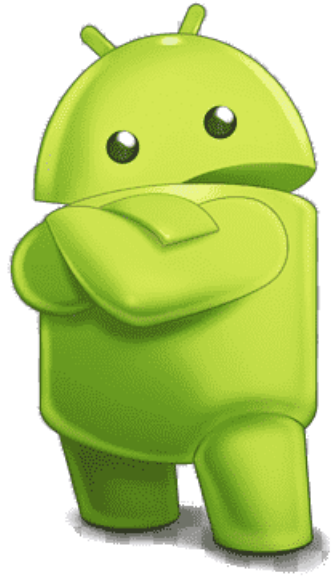
Жизненный цикл деятельности (activity)





Компоненты Android

01

Компоненты Android-приложения

Существуют 4 базовых компонента, которые запускаются и управляются ОС Android и могут существовать сами по себе

1. **Activity** (деятельность или активити): экран приложения с пользовательским интерфейсом (для выполнения одной единственной задачи @Deplecated);
2. **Service** (сервис): исполняемая в фоновом режиме задача (например воспроизведение музыки или скачивание файла);
3. **BroadcastReceiver** (широковещательный приемник): реагирует на сообщения от системы или других приложений, даже если ваше приложение закрыто (например уведомление о заряде батареи или потере сети);
4. **ContentProvider** (менеджер доступа к общим данным): для передачи данных другим приложениям.

Намерения в Android

Каждый из них должен быть указан в файле манифеста и запускается через явное или неявное намерение (Intent)

- **Явное намерение** (Explicit Intent) точно указывает конкретный класс компонента для запуска (например запуск определенной деятельности);
- **Неявное намерение** (Implicit Intent) задает лишь общее действие, позволяя системе Android найти подходящую программу для его выполнения (например поиск всех приложений, которые могут отобразить pdf-файл);
- Сторонние приложения фильтруются на основании указанных поддерживаемых действий в файле AndroidManifest.xml (Intent Filter)

Фильтр намерений

Когда пользователь тапает по иконке приложения, формируется явный intent на основе пары MAIN/LAUNCHER и запускается экран деятельности

AndroidManifest.xml

```
<activity
   android:name = ".FirstActivity"
   android:exported = "true">

   <intent-filter>
      <action android:name = "android.intent.action.MAIN"/>
      <category android:name = "android.intent.category.LAUNCHER"/>
   </intent-filter>

</activity>
```



Деятельность

02

Запуск деятельности

Запустить деятельность можно методом startActivity() с помощью явного (или неявного намерения)

FirstActivity.java

```
Intent intent;

// явно: запустить деятельность SecondActivity
intent = new Intent(this, SecondActivity.class);

// неявно: найти деятельность, которая может заменить браузер
intent = new Intent(Intent.ACTION_VIEW, Uri.parse("https://google.com"));

startActivity(intent);
```

Ответ деятельности

Можно обработать получение результата из иной деятельности

- Пара из методов startActivityForResult() и onActivityResult() объявлены как устаревшие с 2020 года, т.к. они нарушают принцип единственной ответственности, отвечая и за обработку результата (код операции) и за получение данных от второй деятельности;
- Вместо них используется Activity Result API (предлагающий зарегистрировать обработку контрактов, если пользователь их не отклонил). Более того этот механизм доступен не только для компонента деятельности.

Ответ деятельности

Метод launch() запускает контракт, на который мы зарегистрировались методом registerForActivityResult()

FirstActivity.java

```
ActivityResultLauncher<Intent> resultLauncher = registerForActivityResult(
new ActivityResultContracts.StartActivityForResult(),
new ActivityResultCallback<ActivityResult>() {
    @Override
    public void onActivityResult(ActivityResult result) {
        if (result.getResultCode() == Activity.RESULT_OK) {
            // Intent intent = result.getData();
            // useResult(intent);
        }
    }
});
Intent intent = new Intent(this, SecondActivity.class);
resultLauncher.launch(intent);
```

Ответ деятельности

Деятельность возвращает данные методом setResult() с предоставлением кода результата

SecondActivity.java

```
// результат внутрь намерения
Intent intent = new Intent();
intent.putExtra("KEY", "value");

// возврат данных
setResult(RESULT_OK, intent);

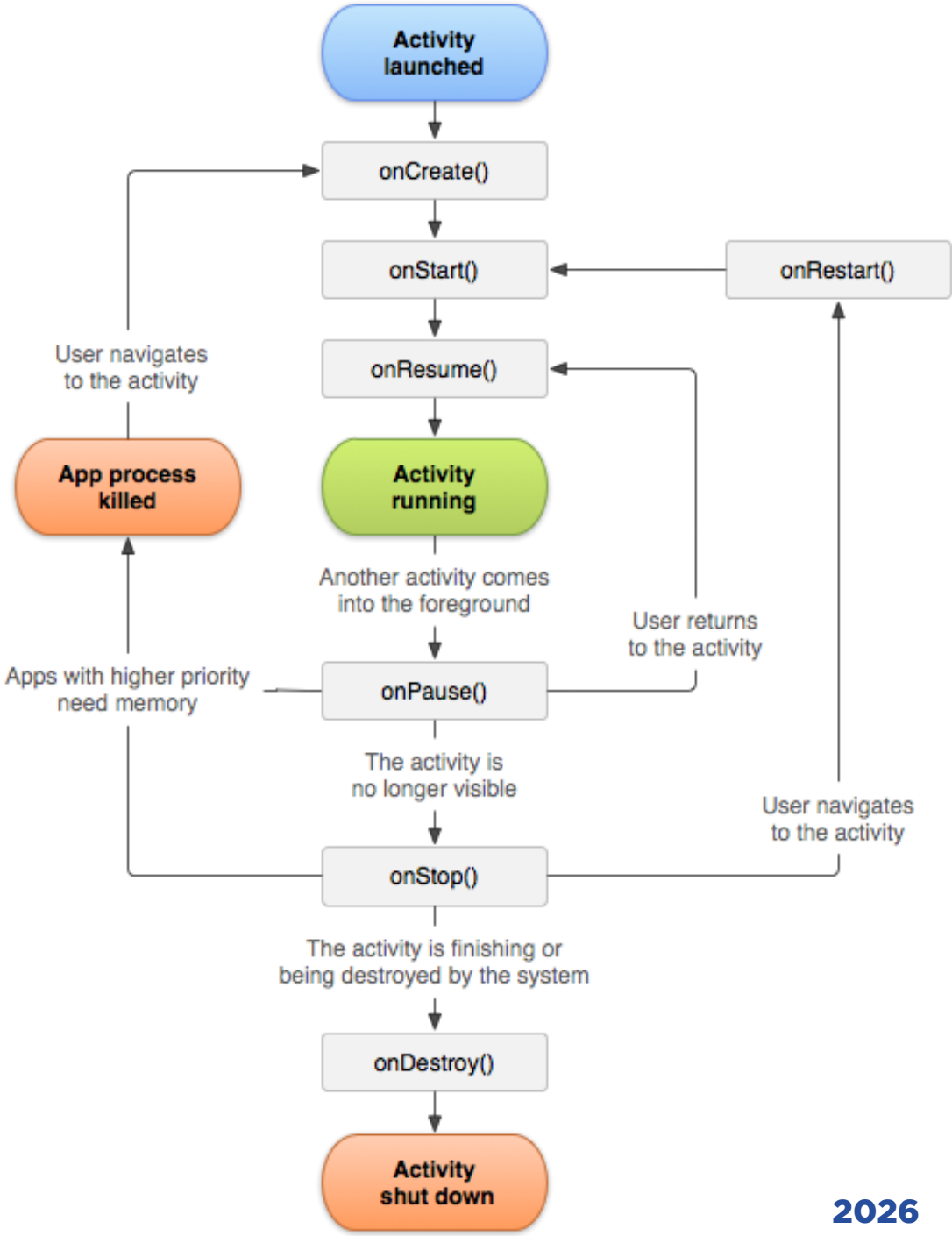
// выключение деятельности
finish();
```

- Код результат будет прочитан инициирующей обмен деятельностью с помощью метода getResultCode();
- Возможны коды RESULT_OK и RESULT_CANCELED (при возврате, свайпе «назад»);
- Данные передаются в паре ключ-значение

Жизненный цикл

Все методы жизненного цикла можно переопределять, но только метод onCreate() переопределить необходимо обязательно

- Внутри метода onCreate() привязывают макет интерфейса, настраивают переменные и инициализируют компоненты;
- В методах onPause/onResume обычно останавливают/включают эффекты анимаций.



onCreate()

Для XML-верстки следует наследоваться от AppCompatActivity, для JetpackCompose – от ComponentActivity

XMLActivity.java

```
public class XMLActivity
    extends AppCompatActivity {

@Override
protected void onCreate
        (Bundle savedInstanceState){
    super.onCreate(savedInstanceState);
    setContentView(R.layout.xml_layout);
}
}
```

ComposeActivity.kt

```
class ComposeActivity:
    ComponentActivity(){

override fun onCreate
        (savedInstanceState: Bundle?){

    super.onCreate(savedInstanceState)
    setContent {Hi()}
}}

@Composable
private fun Hi(){
    Text (text = "Hello world")
}
```